

P3885R0

Add a formatter for `std::error_category`

Published Proposal, 2025-10-19

Author:

[Victor Zverovich](#)

Audience:

SG16

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Motivation

3 Proposal

4 Wording

References

Informative References

§ 1. Introduction

This paper proposes making `std::error_category` formattable using the formatting facility introduced in C++20 (`std::format`).

§ 2. Motivation

`std::error_category` currently has no standard way to be formatted or printed directly. For example none of this compiles:

```
std::cout << std::generic_category();  
std::print("{} ", std::generic_category());
```

Additionally the encoding of `std::error_category::name()` was unspecified making it difficult to use it reliably - this is addressed by [\[P3395\]](#).

§ 3. Proposal

The current paper proposes making `std::error_category` formattable:

```
std::print("{} ", std::generic_category());
```

This prints `generic`.

It will correctly handle width and alignment:

```
std::print("[{:>10}]\n", std::generic_category());
```

Output:

```
[   generic]
```

Similarly to other recent cases, an ostream inserter is not proposed but it is possible to stream the string returned by `std::error_category::name()` or the output of `std::format`.

§ 4. Wording

Add to "Header <system_error> synopsis" [\[system.error.syn\]](#):

```
// [system.error.fmt], formatter
```

```
template<class charT> struct formatter<error_category, charT>;
```

Add a new section "Formatting" [system.error.fmt] under "Class `error_category`" [syserr.errcat]:

```
template<class charT> struct formatter<error_category, charT>
    : formatter<basic_string_view<charT>, charT> {
    template<class FormatContext>
        typename FormatContext::iterator
            format(const error_category& cat, FormatContext& ctx) const;
};
```

```
template<class FormatContext>
    typename FormatContext::iterator
        format(const error_category& cat, FormatContext& ctx) const;
```

Let `name` be `cat.name()` if `charT` is `char` and `cat.name()`, transcoded to wide literal encoding otherwise.

Returns: `formatter<basic_string_view<charT>, charT>::format(name, ctx)`.

§ References

§ Informative References

[P3395]

Victor Zverovich. *Formatter specializations for the standard library*. URL:
<https://isocpp.org/files/papers/P3395R4.html>